



Figure 4: ExampleObject and available methods

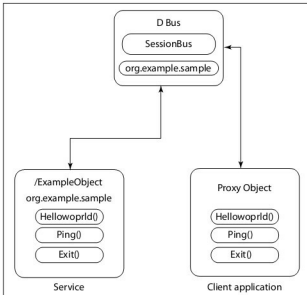


Figure 5: A schematic representation of service-client interaction

```

in_signature=", out_signature=")

def Exit(self):
   .mainloop.quit()

if __name__ == '__main__':
    dbus.mainloop.glib.DBusGMainLoop(set_as_default=True)

    session_bus = dbus.SessionBus()
    name = dbus.service.BusName("org.example.Sample", session_bus)
    object = ExampleObject(session_bus, "/ExampleObject")

   .mainloop = gobject.MainLoop()
    print "Running example dbus service: org.example.Sample"
   .mainloop.run()
  
```

Here we have a class derived from `dbus.service.Object`, which consists of the `HelloWorld()`, `Ping()` and `Exit` functions that are to be exposed through the service. The decorator like `@dbus.service.method("org.example.Sample", in_signature="", out_signature="")` is used to expose these functions. It consists of parameters `in_signature` and `out_signature`, specifying the type of input (parameters to the function) and output (return type). You can refer to Table 1 for the types that are available. For example, 's' specifies string, 'sa' specifies string array, 'i' specifies integer and so on.

Let us now code a D-Bus client (`client.py`) to access methods exported by `org.example.Sample`:

```
#!/usr/bin/env python
```

```

import dbus
bus = dbus.SessionBus()
remote_object = bus.get_object('org.example.Sample', '/ExampleObject')
interface = dbus.Interface(remote_object, 'org.example.Sample')
reply = interface.Ping()
print 'Ping() returns : ' + reply
reply = interface.HelloWorld("GNU/Linux")
  
```

```

print 'HelloWorld() returns: '
for s in reply:
    print s
  
```

If you go through the above code, you can understand that it simply creates a proxy object and an interface to the `org.example.Sample` service. Further, it calls the methods available. You can call it through any type of D-Bus client access method like `dbus-send` tool. Try this:

```

[slynnu@sllynnu-laptop examples]$ dbus-send --session \
--dest=org.example.Sample --print-reply \
/ExampleObject org.example.Sample.Ping
method return sender=:1.326 -> dest=:1.364 reply_serial=2
string "Hi, I am Alive"
  
```

Now, you can open a terminal and execute the service script first and client after that.

On terminal tab 1:

```

[slynnu@sllynnu-laptop examples]$ python example-service.py
Running example dbus service: org.example.Sample
Pinged
GNU/Linux
  
```

One terminal tab 2:

```

[slynnu@sllynnu-laptop examples]$ python example-client.py
Ping() returns : Hi, I am Alive
HelloWorld() returns:
Hello World  dbus-service GNU/Linux
  
```

Hacking other applications with D-Bus

Let us now focus more on the implementation and go through the coding part involving some applications, say, for example, Pidgin. Pidgin is a well-known IM client that a lot of us use to talk to people. We will now work on the D-Bus service interfacing with Pidgin in order to talk with Pidgin:

```

#!/usr/bin/env python
import dbus, subprocess, time
  
```

```

def set_status(message):
    current = purple.PurpleSavedStatusGetType(purple.PurpleSavedStatusGetCurrent())
    status = purple.PurpleSavedStatusNew("", current)
    purple.PurpleSavedStatusSetMessage(status, message)
    purple.PurpleSavedStatusActivate(status)
  
```